

创建启动数据库讲解

林子骥&郭夏辉

2024.10.15

Topics

1. 创建数据库的流程是什么？创建数据库时创建了哪些文件，都有什么作用？
2. DB::Open函数的实现。
3. DBImpl类内都有哪些成员？其作用是什么？
4. DBImpl::Recover函数的实现。
5. DBImpl::NewDB函数的实现。
6. VersionSet::Recover函数的实现。CURRENT的作用是什么？

1.创建数据库的流程是什么？创建数据库时创建了哪些文件，都有什么作用？

准备创建数据库

实例化DBImpl对象

LockFile

创建manifest文件

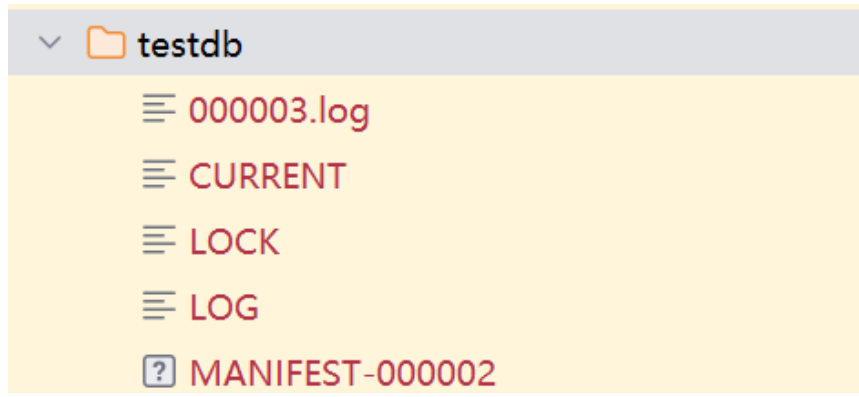
创建CurrentFile

创建LOG文件和创建目录

创建.log文件

创建memTable对象

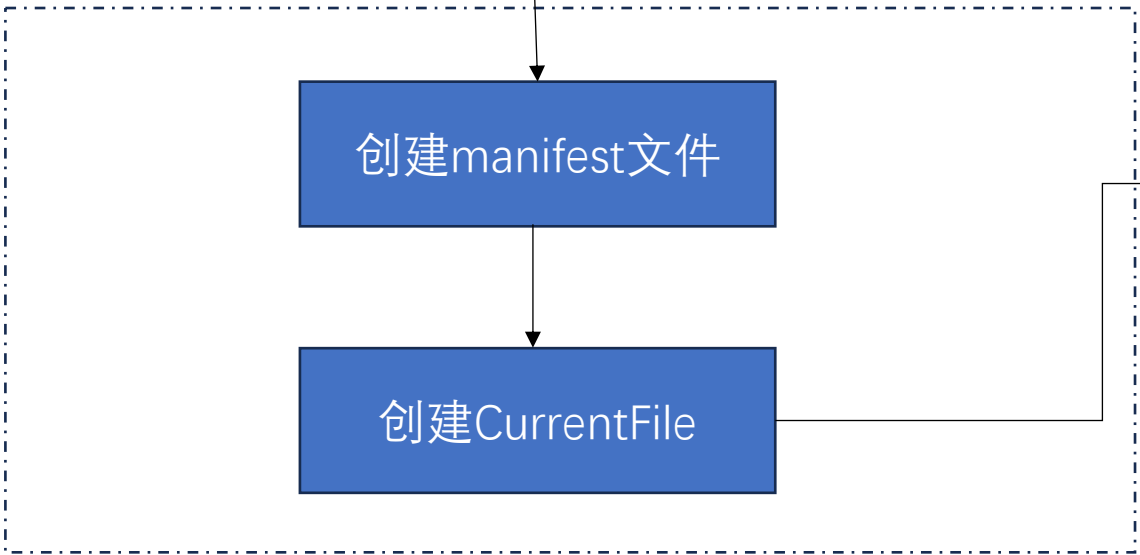
启动后的目录内容:



LOG文件: 是用于记录数据库运行过程中的日志信息, 主要用于调试、监控和记录异常

```
leveldb::PosixEnv::CreateDir env_posix.cc:625
leveldb::SanitizeOptions db_impl.cc:107
leveldb::DBImpl::DBImpl db_impl.cc:131
leveldb::DB::Open db_impl.cc:1516
main db_test1.cc:94
```

NewDB函数



代码解释： 创建目录(testdb)

leveldb::PosixEnv::CreateDir env_posix.cc:625

leveldb::SanitizeOptions db_impl.cc:107

leveldb::DBImpl::DBImpl db_impl.cc:131

leveldb::DB::Open db_impl.cc:1516

main db_test1.cc:94

```
Options SanitizeOptions(const std::string& dbname,
Options result = src;
result.comparator = icmp;
result.filter_policy = (src.filter_policy != nullptr) ? ipolicy : nullptr;
ClipToRange(&result.max_open_files, 64 + kNumNonTableCacheFiles, 50000);
ClipToRange(&result.write_buffer_size, 64 << 10, 1 << 30);
ClipToRange(&result.max_file_size, 1 << 20, 1 << 30);
ClipToRange(&result.block_size, 1 << 10, 4 << 20);
if (result.info_log == nullptr) {
    // Open a log file in the same directory as the db
    src.env->CreateDir(dbname); // In case it does not exist
    src.env->RenameFile(InfoLogFileName(dbname), OldInfoLogFileName(dbname));
    Status s = src.env->NewLogger(InfoLogFileName(dbname), &result.info_log);
    if (!s.ok()) {
        // No place suitable for logging
        result.info_log = nullptr;
    }
}

// Fix user-supplied options to be reasonable
template <class T, class V>
static void ClipToRange(T* ptr, V minvalue, V maxvalue) {
    if (static_cast<V>(*ptr) > maxvalue) *ptr = maxvalue;
    if (static_cast<V>(*ptr) < minvalue) *ptr = minvalue;
}
```

其中，这里的ClipToRange表示作用是**确保用户提供的值在指定的最小值和最大值范围内**。如果值超出了这个范围，函数会将其调整到相应的边界值。

代码解释: LOCK

```
Status LockFile(const std::string& filename, FileLock** lock) override {
    *lock = nullptr;
    //0644 是文件权限，表示文件所有者有读写权限，其他用户只能读
    int fd = ::open(filename.c_str(), O_RDWR | O_CREAT | kOpenBaseFlags, 0644);
    if (fd < 0) {
        return PosixError(filename, errno);
    }

    if (!locks_.Insert(filename)) {
        ::close(fd);
        return Status::IOError("lock " + filename, "already held by process");
    }
}
```

确保只有一个数据库实例访问数据库目录。LOCK 文件通常不包含任何数据内容。其核心功能是作为一个文件锁的机制，通过文件存在和对文件的占用来表示数据库已被某个进程打开。操作系统会保证同一时间只能有一个进程持有该文件的锁。因此，LOCK 文件的内容通常为空白，或者仅包含系统默认的一些占位信息。

代码路径如右图所示:

leveldb::PosixEnv::LockFile env_posix.cc:656

leveldb::DBImpl::Recover db_impl.cc:300

leveldb::DB::Open db_impl.cc:1521

main db_test1.cc:94

解释：MANIFEST

manifest文件就是用来记录这些versionEdit信息的。versionEdit指的是基于旧版本的基础上，变化的内容（例如新增或删除了某些sstable文件）。

例如右图便是一个manifest文件的示意图，其中包含了3条versionEdit记录，每条记录包括（1）新增哪些sst文件（2）删除哪些sst文件（3）当前compaction的下标（4）日志文件编号（5）操作seqNumber等信息。

通过这些信息，leveldb便可以在启动时，基于一个空的version，不断apply这些记录，最终得到一个上次运行结束时的版本信息。

Delta1

Add tables

Del tables

CompPtr

Journal Num

Seq Num

...

Delta2

Add tables

Del tables

CompPtr

Journal Num

Seq Num

...

Delta3

Add tables

Del tables

CompPtr

Journal Num

Seq Num

...

Current解释

```
Status SetCurrentFile(Env* env, const std::string& dbname,
                      uint64_t descriptor_number) {
    // Remove leading "dbname/" and add newline to manifest file name
    // 生成新的 MANIFEST 文件名, 例如 "/data/mydb/MANIFEST-000001"
    std::string manifest = DescriptorFileName(dbname, descriptor_number);
    Slice contents = manifest;
    // 提取文件名部分, 去掉前缀路径部分, 这里变为 "MANIFEST-000001"
    assert(contents.starts_with(dbname + "/"));
    contents.remove_prefix(dbname.size() + 1);
    // 创建一个临时文件, 例如 "/data/mydb/TEMP-000001"
    std::string tmp = TempFileName(dbname, descriptor_number);
    // 将文件名 "MANIFEST-000001\n" 写入临时文件 TEMP-000001
    Status s = WriteStringToFileSync(env, contents.ToString() + "\n", tmp);
    // 如果写入成功, 将临时文件重命名为 "CURRENT"
    if (s.ok()) {
        s = env->RenameFile(tmp, CurrentFileName(dbname));
    }
    if (!s.ok()) {
        env->RemoveFile(tmp);
    }
    return s;
}
```

因为每次leveldb启动时, 都会创建一个新的Manifest文件。因此数据目录可能会存在多个Manifest文件。Current则用来指出哪个Manifest文件才是我们关心的那个Manifest文件。

在 LevelDB 中, 创建临时文件然后重命名 (而不是直接写入或重命名) 是一种常见的**安全策略**, 其目的是为了**确保文件的原子性操作**, 特别是在涉及文件系统时, 防止部分写入、系统崩溃或程序中断造成的不一致性或数据损坏

leveldb::SetCurrentFile filename.cc:124

leveldb::DBImpl::NewDB db_impl.cc:209

leveldb::DBImpl::Recover db_impl.cc:309

leveldb::DB::Open db_impl.cc:1521

main db_test1.cc:94

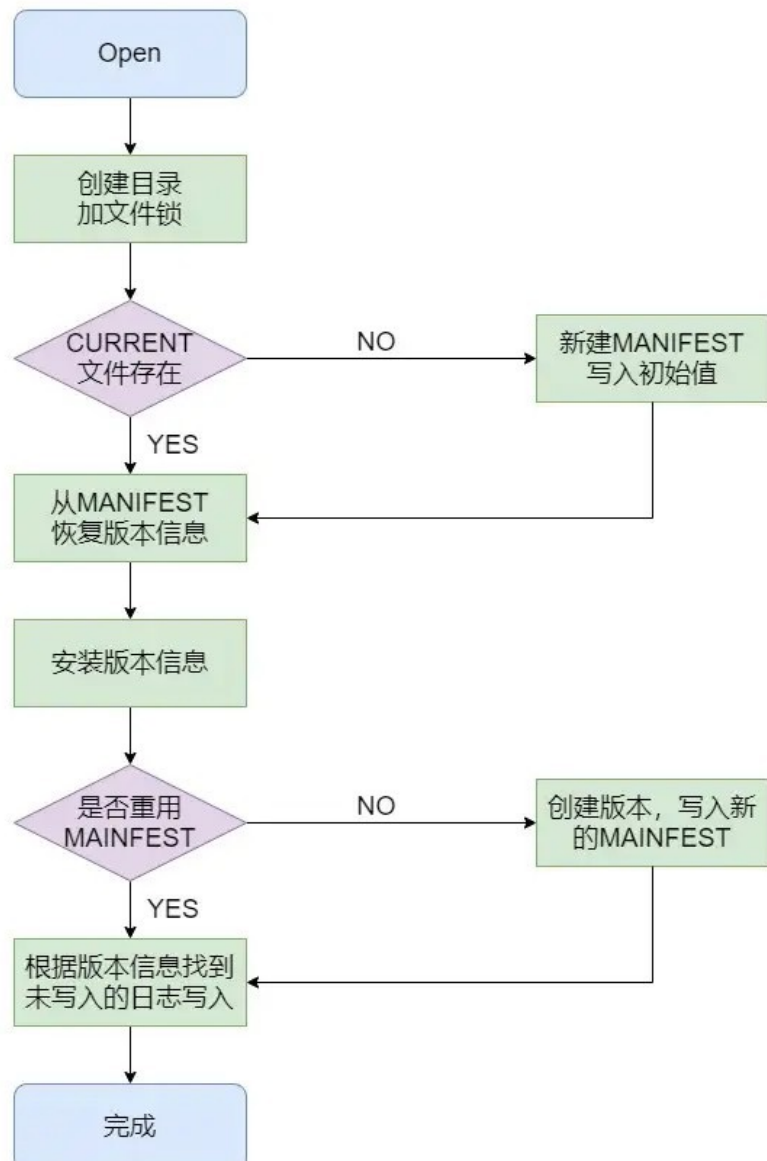
.log文件

```
Status DB::Open(const Options& options, const std::string& dbname, DB** dbptr) {
    bool save_manifest = false;
    Status s = impl->Recover(&edit, &save_manifest);
    if (s.ok() && impl->mem_ == nullptr) {
        // Create new log and a corresponding memtable.
        uint64_t new_log_number = impl->versions_->NewFileNumber();
        WritableFile* lfile;
        s = options.env->NewWritableFile(LogFileName(dbname, new_log_number),
                                         &lfile);

        if (s.ok()) {
            edit.SetLogNumber(new_log_number);
            impl->logfile_ = lfile;
            impl->logfile_number_ = new_log_number;
            impl->log_ = new log::Writer(lfile);
            impl->mem_ = new MemTable(impl->internal_comparator_);
            impl->mem_->Ref();
        }
    }
}
```

.log文件：用于记录未持久化的数据，用于崩溃恢复

DB::Open函数的实现



// db/db_impl.cc

```
Status DB::Open(const Options& options, const std::string& dbname, DB** dbptr) {  
    *dbptr = nullptr;
```

```
    DBImpl* impl = new DBImpl(options, dbname);
```

```
    VersionEdit edit;
```

```
    bool save_manifest = false;
```

```
    // 调用DBImpl::Recover完成MANIFEST的加载和故障恢复
```

```
    // 包含目录上锁, 新建manifest和调整current
```

```
    Status s = impl->Recover(&edit, &save_manifest);
```

```
    if (s.ok() && impl->mem_ == nullptr) {
```

```
        // 创建日志和相应的MemTable
```

```
    }
```

```
    if (s.ok() && save_manifest) {
```

```
        // 如果需要重写MANIFEST文件, 那么做一个版本变更, 这里面会创建一个新的MANIFEST
```

```
        // 将当前的版本信息写入, 然后将edit的内容写入。
```

```
        edit.SetLogNumber(impl->logfile_number_);
```

```
        s = impl->versions_->LogAndApply(&edit, &impl->mutex_);
```

```
    }
```

```
    if (s.ok() && save_manifest) {
```

```
        edit.SetPrevLogNumber(0); // No older logs needed after recovery.
```

```
        edit.SetLogNumber(impl->logfile_number_);
```

```
        s = impl->versions_->LogAndApply(&edit, &impl->mutex_);
```

```
    }
```

```
    if (s.ok()) {
```

```
        impl->RemoveObsoleteFiles();
```

```
        impl->MaybeScheduleCompaction();
```

```
    }
```

```
    impl->mutex_.Unlock();
```

```
    return s;
```

```
}
```

```

void DBImpl::RemoveObsoleteFiles() {
    std::set<uint64_t> live = pending_outputs_;
    versions_>AddLiveFiles(&live);

    std::vector<std::string> filenames;
    env_>GetChildren(dbname_, &filenames); // Ignoring errors on purpose
    uint64_t number;
    FileType type;
    std::vector<std::string> files_to_delete;
    for (std::string& filename : filenames) {
        if (ParseFileName(filename, &number, &type)) {
            bool keep = true;
            switch (type) {
                case kLogFile:
                    keep = ((number >= versions_>LogNumber()) || (number == versions_>PrevLogNumber()));
                    break;
                case kDescriptorFile:
                    keep = (number >= versions_>ManifestFileNumber());
                    break;
                case kTableFile:
                    keep = (live.find(number) != live.end());
                    break;
                case kTempFile:
                    // Any temp files that are currently being written to must
                    // be recorded in pending_outputs_, which is inserted into "live"
                    keep = (live.find(number) != live.end());
                    break;
                case kCurrentFile:
                case kDBLockFile:
                case kInfoLogFile:
                    keep = true;
                    break;
            }
        }
    }
    for (const std::string& filename : files_to_delete) {
        env_>RemoveFile(dbname_ + "/" + filename);
    }
}

```

testdb

- 000003.log
- CURRENT
- LOCK
- LOG
- MANIFEST-000001
- MANIFEST-000002

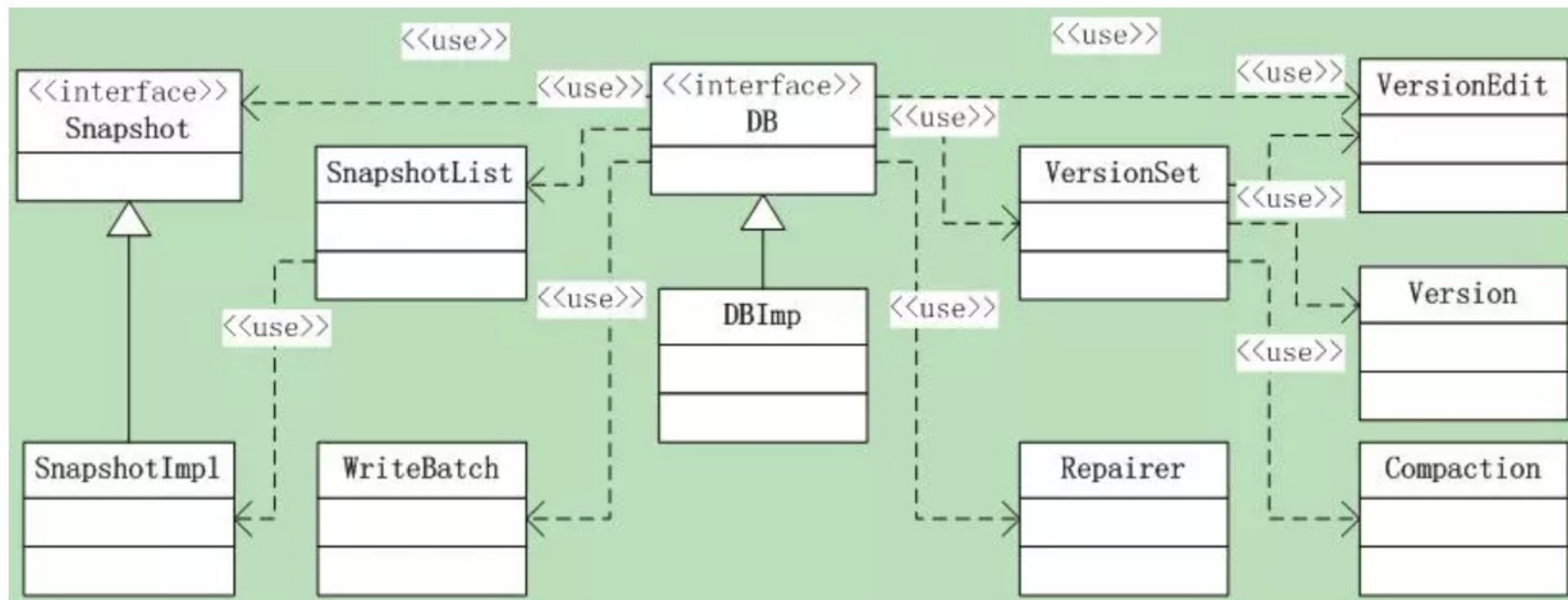
这里我们发现了number的判定包括**两个比较**。

这是因为LogNumber()表示当前正在使用的日志文件的编号。它用于记录当前正在写入内存表的操作。

PrevLogNumber()表示之前使用的日志文件编号。这个编号对应的是上一个日志文件，在特定的情况下（如系统崩溃或进程异常终止），这个日志文件可能还未完全处理完，因此也需要保留。

DBImpl类内都有哪些成员？其作用是什么？

Dbimpl是对db类的继承



```
// Filter policy wrapper that converts from internal keys to user keys
class InternalFilterPolicy : public FilterPolicy {
```

```
// Constant after construction
```

```
Env* const env_; // 环境, 封装了系统相关的文件操作、线程等等
```

```
const InternalKeyComparator internal_comparator_; // key comparator
```

```
const InternalFilterPolicy internal_filter_policy_; // filter policy
```

```
const Options options_; // options_.comparator == &internal_comparator_
```

```
const bool owns_info_log_; // 用户定制相关
```

```
const bool owns_cache_; // 用户定制相关
```

```
const std::string dbname_;
```

这里的owns_info_log, 主要在用户需要自定义行为时有用。例如用户可能希望使用特定的日志格式或内容, 满足业务的定制需求, 比如日志中包含额外的元数据、时间戳、请求 ID 等信息。这样就需要在外部进行资源管理操作(delete).
Cache同理。

```
namespace rocksdb {
DBImpl::~DBImpl() {
    env_>UnlockFile(lock: db_lock_);
}

delete versions_;
if (mem_ != nullptr) mem_>Unref();
if (imm_ != nullptr) imm_>Unref();
delete tmp_batch_;
delete log_;
delete logfile_;
delete table_cache_;

if (!owns_info_log_) {
    delete options_.info_log;
}
if (owns_cache_) {
    delete options_.block_cache;
}
}
```



```
// table_cache_ provides its own synchronization
```

```
TableCache* const table_cache_;
```

```
// Lock over the persistent DB state.
```

```
FileLock* db_lock_;
```

table_cache_ 负责管理对已打开的 SSTable 文件的缓存。通过缓存 SSTable 元数据和块信息来提高读取效率。在该实例中，它的唯一作用就是传递参数给 versions。

Dblock: 在之前已经提到过，用于单线程打开数据库

```
Status TableCache::Get(const ReadOptions& options, uint64_t file_number,
```

Function Get leveldb::TableCache

2 us

Project Files

```
Cache::Handle* hand
```

```
Status s = FindTabl
```

```
if (s.ok()) {
```

table_cache.h (43: 10)

Status Get(const ReadOptions& options, uint64_t file_number,

version_set.cc (354: 45)

state->s = state->vset->table_cache_->Get(*state->options, f->number,

```
DBImpl::DBImpl(const Options& raw_options, const std::string& dbname)
```

```
: env_(raw_options.env),
```

```
internal_comparator_(c: raw_options.comparator),
```

```
internal_filter_policy_(p: raw_options.filter_policy),
```

```
options_(SanitizeOptions(dbname, icmp: &internal_comparator_,
```

```
ipolicy: &internal_filter_policy_, src: raw_options)),
```

```
owns_info_log_(options_.info_log != raw_options.info_log),
```

```
owns_cache_(options_.block_cache != raw_options.block_cache),
```

```
dbname_(dbname),
```

```
table_cache_(new TableCache(dbname: dbname_, options: options_, entries: TableCacheSize(sanitized_options: options_))),
```

```
db_lock_(nullptr),
```

```
shutting_down_(i: false),
```

```
background_work_finished_signal_(mu: &mutex_),
```

```
mem_(nullptr),
```

```
imm_(nullptr),
```

```
has_imm_(i: false),
```

```
logfile_(nullptr),
```

```
logfile_number_(0),
```

```
log_(nullptr),
```

```
seed_(0),
```

```
tmp_batch_(new WriteBatch),
```

```
background_compaction_scheduled_(false),
```

```
manual_compaction_(nullptr),
```

```
versions_(new VersionSet(dbname: dbname_, options: &options_, table_cache: table_cache_,
```

```
&internal_comparator_)) {}
```

```

// State below is protected by mutex_
port::Mutex mutex_;
std::atomic<bool> shutting_down_; // 表示数据库实例是否正在关闭中
port::CondVar background_work_finished_signal_ GUARDED_BY(mutex_);
MemTable* mem_;
MemTable* imm_ GUARDED_BY(mutex_); // Memtable being compacted
std::atomic<bool> has_imm_; // So bg thread can detect non-null imm_

```

WritableFile* logfile_;//logfile_ 是一个指向底层文件句柄的指针, 指向当前正在使用的

uint64_t logfile_nu

log::Writer* log_;/

uint32_t seed_ GUAR

Field logfile_ leveldb::DBImpl 10 usages

Project Files

db_impl.cc (142: 7) logfile_(nullptr),

db_impl.cc (170: 10) delete logfile_;

db_impl.cc (473: 12) assert(logfile_ == nullptr);

>NewAppendableFile(fname

= new log::Writer(logfile_, lfil

= logfile_->Sync();

gfile_->Close();

logfile_;

e_ = lfile;

>logfile_ = lfile;

Shutting_down:表示数据库实例是否正在关闭中。

触发场景: 数据库关闭过程中, 调用~DBImpl(),设置为True;后台压缩进程定期检查, 若为True, 提前结束任务, 安全退出。

note: 基于线程安全, 它的相关原子操作都为std::memory_order_release或std::memory_order_acquired

Background_work_finished_signal:用于唤醒正在等待的compaction线程。

其余的成员变量就比较熟悉了, 在这里不继续赘述, 简单讲一讲。

WritableFile* logfile_; //logfile_ 是一个指向底层文件句柄的指针，指向当前正在使用的日志文件。

uint64_t logfile_number_ GUARDED_BY(mutex_);

log::Writer* log_; //封装日志写入逻辑的对象。log_ 负责将数据（写操作）组织成特定格式并将其写入到 logfile_ 指向的文件中。

WritableFile* logfile_;//logfile_ 是一个指向底层文件句柄的指针，指向当前正在使用的

uint64_t logfile_nu

log::Writer* log_;

uint32_t seed_ GUAR

// Queue of writers

std::deque<Writer*>

WriteBatch* tmp_bat

SnapshotList snapsh

// Set of table fil

// part of ongoing

std::set<uint64_t>

// Has a background

Field logfile_ leveldb::DBImpl

10 usages



db_impl.cc (142: 7)

logfile_(nullptr),

db_impl.cc (170: 10)

delete logfile_;

db_impl.cc (473: 12)

assert(logfile_ == nullptr);

db_impl.cc (478: 41)

env_>NewAppendableFile(fname

db_impl.cc (480: 30)

log_ = new log::Writer(logfile_, lf

db_impl.cc (1238: 18)

status = logfile_>Sync();

db_impl.cc (1379: 11)

s = logfile_>Close();

db_impl.cc (1390: 14)

delete logfile_;

db_impl.cc (1392: 7)

logfile_ = lf;

db_impl.cc (1519: 13)

impl->logfile_ = lf;

uint64_t logfile_number_ GUARDED_BY(mutex_);

log::Writer* lo

uint32_t seed_

Field logfile_number_ leveldb::DBImpl

6 usages



db_impl.cc (143: 7)

logfile_number_(0),

db_impl.cc (481: 7)

logfile_number_ = log_number;

db_impl.cc (567: 23)

edit.SetLogNumber(logfile_number

db_impl.cc (1393: 7)

logfile_number_ = new_log_numbe

db_impl.cc (1520: 13)

impl->logfile_number_ = new_log_

db_impl.cc (1528: 25)

edit.SetLogNumber(impl->logfile_n

// Queue of wri

std::deque<Writ

WriteBatch* tmp

SnapshotList sn

// Set of table

// part of onpa

void DBImpl::CompactMemTable() {

edit.SetPrevLogNumber(num: 0);//这意味着所有在旧日志文件 (PrevLogNumber) 中存储的数据已经被处理或写入到 S

edit.SetLogNumber(num: logfile_number_); // Earlier logs no longer needed

s = versions_>LogAndApply(&edit, mu:&mutex_);

}

Status VersionSet::LogAndApply(VersionEdit* edit, port::Mutex* mu) {

if (s.ok()) {

AppendVersion(v);

log_number_ = edit->log_number_;

Logfile_number_变化的场景:

1.初始化, 包括open db和new操作

2.recovery_file时重新赋值;

3.MakeRoomForWrite,创建新mem.

用处: 传递参数给version_edit

```
// Queue of writers.
std::deque<Writer*> writers_ GUARDED_BY(mutex_);
WriteBatch* tmp_batch_ GUARDED_BY(mutex_);
```

```
SnapshotList snapshots_ GUARDED_BY(mutex_);
```

尾插法维护，在该实例中，主要用于在compaction操作中获取最老的seq。

```
// Set of table files to protect from deletion because they are
// part of ongoing compactions.
```

```
std::set<uint64_t> pending_outputs_ GUARDED_BY(mutex_); // 待compaction的文件列表，保护以防误删
```

Pending_out触发场景：在往level0层写入与打开相关compaction文件时维护。

```
// Has a background compaction been scheduled or is running?
```

```
bool background_compaction_scheduled_ GUARDED_BY(mutex_); // 是否有后台compaction在调度或者运行?
```

```
ManualCompaction* manual_compaction_ GUARDED_BY(mutex_); // 手动compaction信息
```

```
VersionSet* const versions_ GUARDED_BY(mutex_); // 多版本DB文件，又一个庞然大物
```

```
// Have we encountered a background error in paranoid mode?
```

```
Status bg_error_ GUARDED_BY(mutex_); // paranoid mode下是否有后台错误?
```

```
CompactionStats stats_[config::kNumLevels] GUARDED_BY(mutex_); // compaction状态
```

```
// Information for a manual compaction
```

```
struct ManualCompaction {
```

```
    int level;
```

```
    bool done;
```

```
    const InternalKey* begin; // null means beginning of key range
```

```
    const InternalKey* end; // null means end of key range
```

```
    InternalKey tmp_storage; // Used to keep track of compaction progress
```

```
};
```

Bg_error触发场景:例如当后台执行 SST 文件的 compaction 操作时，若发生文件读取或写入错误。如果后台发生与磁盘 I/O 相关的问题（如磁盘损坏、空间不足等），这些错误也会被捕获。还有读写权限等问题。

States主要用于对合并过程的性能监控和优化.例如，发现某一层级合并频繁、耗时长或读取/写入的数据量过大时，可以调节合并频率或调整相关的参数(如 SST 文件大小)，从而优化写入性能。

Versions_介绍:

Version

Version里面保存了各个level下每个sstable的FileMetaData
FileMetaData里存放了filenumber, filesize, smallestkey, largestkey等信息

files_

Level-0	FileMetaData	FileMetaData	...	FileMetaData
Level-1	FileMetaData	FileMetaData	...	FileMetaData
Level-...	...			
Level-n	FileMetaData	FileMetaData	...	FileMetaData

VersionEdit

VersionEdit里面保存了此次compact新生成的sstable所处level和MetaData
同时保存了需要被删除的sstable（即被compact的sstable）所处level和filenumber

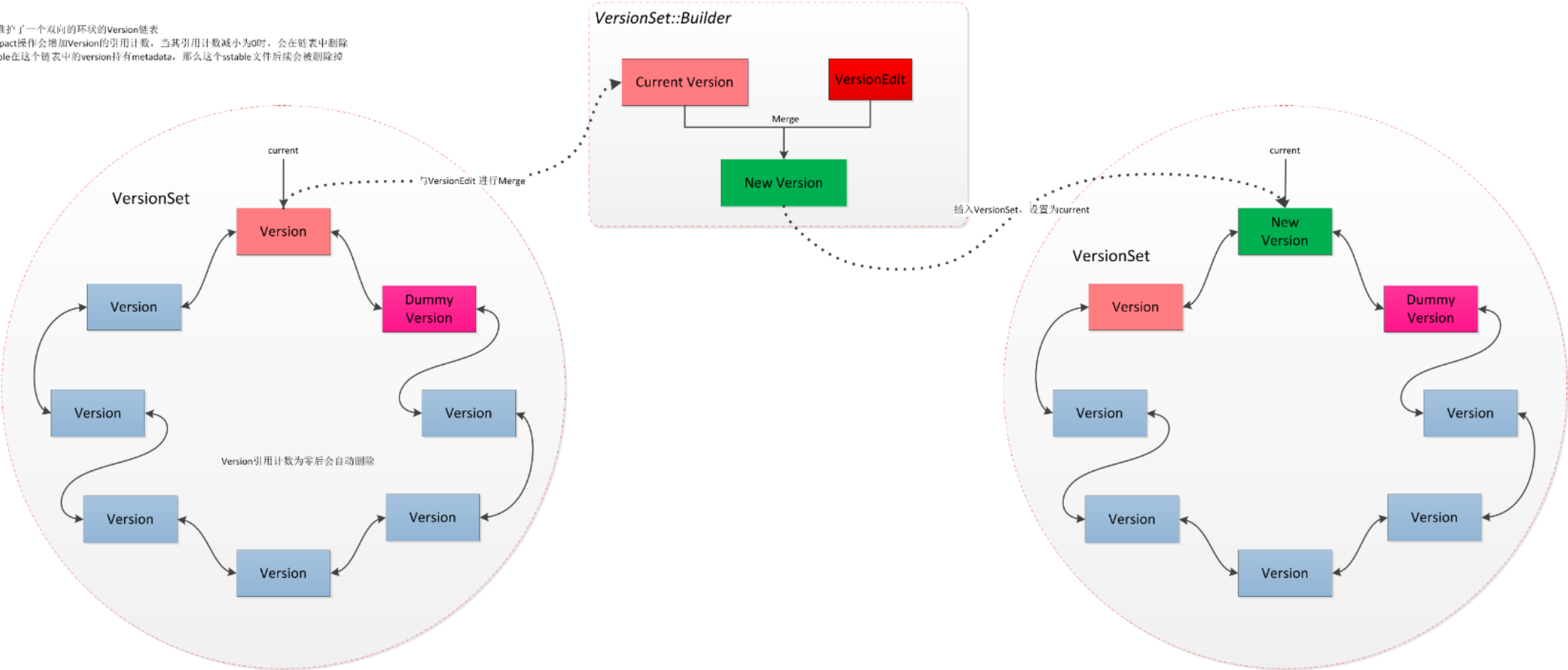
deleted_files

pair<level, filenumber>	...	pair<level, filenumber>
-------------------------	-----	-------------------------

new_files_

pair<Level, FileMetaData>	...	pair<Level, FileMetaData>
---------------------------	-----	---------------------------

VersionSet里维护了一个双向的环状的Version链表
读操作或compact操作会增加Version的引用计数，当其引用计数减小为0时，会在链表中删除
如果某个sstable在这个链表中的version持有metadata，那么这个sstable文件后续会被删除掉



新的疑问：这么多变量被mutex保护，是否会影响读写性能。


```

Status DBImpl::Get(const ReadOptions& options, const Slice& key,
Version* current = versions_->current();
mem->Ref();
if (imm != nullptr) imm->Ref();
current->Ref();

bool have_stat_update = false;
Version::GetStats stats;

// Unlock while reading from files and memtables
{
    mutex_.Unlock();
    // First look in the memtable, then in the immutable memtable (if any).
    LookupKey lkey(user_key: key, sequence: snapshot);
    if (mem->Get(key: lkey, value, &s)) {
        // Done
    } else if (imm != nullptr && imm->Get(key: lkey, value, &s)) {
        // Done
    } else {
        s = current->Get(options, key: lkey, val: value, &stats);
        have_stat_update = true;
    }
    mutex_.Lock();
}

if (have_stat_update && current->UpdateStats(stats)) {
    MaybeScheduleCompaction();
}
mem->Unref();
if (imm != nullptr) imm->Unref();
current->Unref();

```

```

Status DBImpl::Write(const WriteOptions& options, WriteBatch* updates) {
    uint64_t last_sequence = versions_->LastSequence();
    Writer* last_writer = &w;
    if (status.ok() && updates != nullptr) { // nullptr batch is for compactions
        WriteBatch* write_batch = BuildBatchGroup(&last_writer);
        WriteBatchInternal::SetSequence(batch: write_batch, seq: last_sequence + 1);
        last_sequence += WriteBatchInternal::Count(batch: write_batch);

        // Add to log and apply to memtable. We can release the lock
        // during this phase since &w is currently responsible for logging
        // and protects against concurrent loggers and concurrent writes
        // into mem_.
        {
            mutex_.Unlock();
            status = log_->AddRecord(slice: WriteBatchInternal::Contents(batch: write_batch));
            bool sync_error = false;
            if (status.ok() && options.sync) {...}
            if (status.ok()) {
                status = WriteBatchInternal::InsertInto(batch: write_batch, memtable: mem_);
            }
            mutex_.Lock();
            if (sync_error) {...}
        }
        if (write_batch == tmp_batch_) tmp_batch_->Clear();

        versions_->SetLastSequence(s: last_sequence);
    }

    while (true) {
        Writer* ready = writers_.front();
    }
}

```

答案是不会的：锁的持有主要涉及相关元数据的处理，在读写某一key时进行IO操作，或者对缓存操作时，并不持有锁，依靠原子性保证线程安全。而另一可能的IO操作，合并操作(MaybeScheduleCompaction)，它将在后台运行，不会持有锁。

DBImpl::Recover函数的实现

- 创建数据库目录;
- 对这个数据库里面的LOCK文件加文件锁;

LevelDB单进程多线程，需要保证每次只有一个进程能够打开数据库 方式：文件锁

- 如果数据库不存在，说明是新打开的DB，调用DBImpl::NewDB创建新数据库
- 如果是重启DB，调用VersionSet::Recover来读取MANIFEST中每次版本的更改，恢复当前版本信息
- 根据版本信息，搜索数据库目录，找到关闭时没有写入到SSTable的日志，按日志写入顺序逐个恢复日志数据。
- DBImpl::RecoverLogFile会创建一个MemTable，开始读取日志信息，将日志的数据插入到MemTable并根据需要调用DBImpl::WriteLevel0Table将MemTable写入到SSTable中

DBImpl::Recover函数的实现

(db/db_impl.cc)

- Step1 创建目录,
- 然后尝试获取它的文件锁, 失败则返回

```
env_ -> CreateDir(dbname_); // 创建数据库目录
assert(db_lock_ == nullptr);
// 加文件锁, 防止其他进程进入
Status s = env_ -> LockFile(LockFileName(dbname_), &db_lock_);
if (!s.ok()) {
    return s;
}
```

- Step2
根据CURRENT文件是否存在
以及option参数执行检查。

```
if (!env_ -> FileExists(CurrentFileName(dbname_))) {
    // 如果CURRENT文件不存在, 说明需要新创建数据库
    if (options_.create_if_missing) {
        Log(options_.info_log, "Creating DB %s since it was missing.",
            dbname_.c_str());
        s = NewDB(); // 新创建数据库
        if (!s.ok()) {
            return s;
        }
    } else {
        return Status::InvalidArgument(
            dbname_, "does not exist (create_if_missing is false)");
    }
} else {
    if (options_.error_if_exists) {
        return Status::InvalidArgument(dbname_,
            "exists (error_if_exists is true)");
    }
}
```

- Step3
调用VersionSet的Recover()函数
就是从文件中恢复数据。
如果成功则执行第四步。

```
// 读取MANIFEST文件进行版本信息的恢复
s = versions_ -> Recover(save_manifest);
if (!s.ok()) {
    return s;
}
```

DBImpl::Recover函数的实现 (db/db_impl.cc)

- Step4 尝试从所有比manifest文件中记录的log要新的log文件中恢复。
- Step4-1先找出所有满足比manifest文件记录的log编号更新的log文件。

```
SequenceNumber max_sequence(0);
const uint64_t min_log = versions_>LogNumber();
const uint64_t prev_log = versions_>PrevLogNumber();
std::vector<std::string> filenames;
s = env_>GetChildren(dbname_, &filenames);
if (!s.ok()) {
    return s;
}
std::set<uint64_t> expected;
versions_>AddLiveFiles(&expected);
uint64_t number;
FileType type;
std::vector<uint64_t> logs;
for (size_t i = 0; i < filenames.size(); i++) {
    if (ParseFileName(filenames[i], &number, &type)) {
        expected.erase(number);
        if (type == kLogFile && ((number >= min_log) || (number == prev_log)))
            logs.push_back(number);
    }
}
```

之所以要进行Step4的理由：
前一个版本可能会添加新的log文件，却没有记录在manifest中。最后如果发现大于原信息记录的log编号的log文件，则需要回放log，更新db数据。回放期间db可能会dump新的level 0文件，因此需要把db元信息的变动记录到edit中返回。

有趣的小细节：为什么判断条件里已经有了`number >= min_log`(原记录的log编号)为什么`number == prev_log`也行？

事实上，函数`PrevLogNumber()`已经不再用了，仅为了兼容老版本。

DBImpl::Recover函数的实现 (db/db_impl.cc)

- Step4-2 排序，依次回放log。并把sstable文件的变动追加到edit中返回

```
// 从WAL中恢复memtable
// Recover in the order in which the logs were generated
std::sort(logs.begin(), logs.end());
// 逐个恢复日志的内容
for (size_t i = 0; i < logs.size(); i++) {
    s = RecoverLogFile(logs[i], (i == logs.size() - 1), save_manifest, edit,
        &max_sequence);
    if (!s.ok()) {
        return s;
    }
    // 前一版可能在生成该log编号后没有记录在MANIFEST中,所以这里我们手动更新VersionSet中的文件编号计数器
    // The previous incarnation may not have written any MANIFEST
    // records after allocating this log number.  So we manually
    // update the file number allocation counter in VersionSet.
    versions_>MarkFileNumberUsed(logs[i]);
}
```

这里要手动更新
VersionSet中的
文件编号计数器

- Step4-3更新VersionSet的sequence

```
if (versions_>LastSequence() < max_sequence) {
    versions_>SetLastSequence(max_sequence);
}
```

DBImpl::NewDB函数的实现 (db/db_impl.cc)

- Step1 产生DB元信息

```
VersionEdit new_db;
```

```
// 保存比较器的名称，下次打开时需要用相同的名称打开
```

```
new_db.SetComparatorName(user_comparator()->Name());
```

```
new_db.SetLogNumber(0); // 分配日志文件的编号为0
```

```
new_db.SetNextFile(2); // 下一个待分配的文件编号是2，因为1分配给了MANIFEST文件
```

```
new_db.SetLastSequence(0);
```

- 下一个待分配的文件编号是多少?
- 2 因为1分配给了MANIFEST文件。

- Step2 产生MANIFEST文件
将db元信息写入它。

Step3 如果成功，就把
MANIFEST文件名写入到
CURRENT文件中

```
if (s.ok()) {  
    // Make "CURRENT" file that points to  
    // 让CURRENT文件指向这个MANIFEST文件  
    s = SetCurrentFile(env_, dbname_, 1);  
} else {  
    env_->RemoveFile(manifest);  
}
```

```
const std::string manifest = DescriptorFileName(dbname_, 1);  
WritableFile* file;  
Status s = env_->NewWritableFile(manifest, &file);  
if (!s.ok()) {  
    return s;  
}  
  
log::Writer log(file);  
std::string record;  
new_db.EncodeTo(&record);  
s = log.AddRecord(record);  
if (s.ok()) {  
    s = file->Sync();  
}  
if (s.ok()) {  
    s = file->Close();  
}  
}  
delete file;
```

DBImpl::NewDB函数的实现

(db/db_impl.cc)

DBImpl::NewDB函数的实现 (db/db_impl.cc)

- DBImpl::NewDB出人意料的简单
- 一个新的数据库没有任何数据，所以不需要日志和SSTable，只需要有一个MANIFEST文件，包含一些元数据就行了。
- 最后CURRENT要指向新创建的MANIFEST文件。

VersionSet::Recover函数的实现 (db/version_set.cc)

概括来说，VersionSet::Recover会：

- 通过读取 CURRENT 文件，找到当前正在使用的 MANIFEST 文件
- 读取 MANIFEST 文件，将数据库恢复到上次关闭前的状态

为什么要这么做？

- 恢复出版本信息后，安装这个版本，那么数据库的元数据就恢复到了关闭时候的状态，这个数据库才准备好了可以读写了。

我们恢复的是数据库哪些状态？

- 我们需要知道数据库里有哪些SSTable文件，每个文件处于哪个Level，当前日志的编号等信息。

VersionSet::Recover函数的实现 (db/version_set.cc)

- Step1 读取CURRENT文件的内容，获取当前使用的MANIFEST文件。

```
struct LogReporter : public log::Reader::Reporter {
    Status* status;
    void Corruption(size_t bytes, const Status& s) override {
        if (this->status->ok()) *this->status = s;
    }
};

// Read "CURRENT" file, which contains a pointer to the current manifest file
// 读取 CURRENT 文件的内容，存放到 std::string current 中。
std::string current;
Status s = ReadFileToString(env_, CurrentFileName(dbname_), &current);
if (!s.ok()) {
    return s;
}

// 如果 CURRENT 文件为空，或者没有以 '\n' 结尾，说明 CURRENT 文件有问题，返回失败。
if (current.empty() || current[current.size() - 1] != '\n') {
    return Status::Corruption("CURRENT file does not end with newline");
}

// 去掉 current 中的 '\n'。
current.resize(current.size() - 1);

// current 中存放的是 MANIFEST 的文件名，例如 MANIFEST-000001。dscname 为 MANIFEST 的完整路径。
std::string dscname = dbname_ + "/" + current;
SequentialFile* file;
s = env_->NewSequentialFile(dscname, &file); // 以顺序读取的方式打开 MANIFEST 文件。
if (!s.ok()) {
    if (s.IsNotFound()) {
        return Status::Corruption("CURRENT points to a non-existent file",
            s.ToString());
    }
}
return s;
}
```

- 需要关注current的文件的格式问题
- 文件名要转化为完整路径才能打开

VersionSet::Recover函数的实现 (db/version_set.cc)

- Step2 读取MANIFEST文件，将里面的VersionEdit读取应用到一个builder里

```
{
// 构造一个 reader，用于读取 MANIFEST 文件。
LogReporter reporter;
reporter.status = &s;
log::Reader reader(file, &reporter, true /*checksum*/, 0 /*initial_offset*/);
Slice record;
std::string scratch;
while (reader.ReadRecord(&record, &scratch) && s.ok()) {
  ++read_records;
  // 将 record 解码成 VersionEdit。
  VersionEdit edit;
  s = edit.DecodeFrom(record);
  if (s.ok()) {
    // 检查解码出来的 VersionEdit.comparator 与当前数据库的 comparator 是否一致。
    if (edit.has_comparator_ &&
        edit.comparator_ != icmp_.user_comparator()->Name()) { ...
  }
  // 将 VersionEdit 应用到 builder 中。
  if (s.ok()) {
    builder.Apply(&edit);
  }
  // 记录最新状态下的 WAL 编号。
  if (edit.has_log_number_) { ...
  // 记录最新状态下的 prev WAL 编号。
  if (edit.has_prev_log_number_) { ...
  // 记录最新状态下的 next_file 编号。
  if (edit.has_next_file_number_) { ...
  // 记录最新状态下的 last_sequence 编号。
  if (edit.has_last_sequence_) { ...
}
```

- 读取 MANIFEST 文件中的每一条 Record
- 把 Record 解码成一个 VersionEdit
- 然后调用 builder.Apply(&edit) 将 edit 应用到 builder 中。

VersionSet::Recover函数的实现 (db/version_set.cc)

- 读取之后当然要关闭文件

```
// 读取 MANIFEST 文件完毕，关闭文件。
```

```
delete file;
```

```
file = nullptr;
```

```
if (s.ok()) {
```

```
    if (!have_next_file) {
```

```
        s = Status::Corruption("no meta-nextfile entry in descriptor");
```

```
    } else if (!have_log_number) {
```

```
        s = Status::Corruption("no meta-lognumber entry in descriptor");
```

```
    } else if (!have_last_sequence) {
```

```
        s = Status::Corruption("no last-sequence-number entry in descriptor");
```

```
    }
```

```
    if (!have_prev_log_number) {
```

```
        prev_log_number = 0;
```

```
    }
```

```
// 标记 next_file_number_ 和 log_number 都被占用了。
```

```
MarkFileNumberUsed(prev_log_number);
```

```
MarkFileNumberUsed(log_number);
```

```
}
```


VersionSet::Recover函数的实现 (db/version_set.cc)

- Step3 基于 MANIFEST 里的 VersionEdit列表, 构造一个新的 Version, 这一步尤为重要。

```
if (s.ok()) {
    // 核心代码
    // 基于 MANIFEST 里的 VersionEdit列表, 构造一个新的 Version。
    Version* v = new Version(this); // 创建一个新版本版本
    builder.SaveTo(v); // 将builder里面的内容应用到新版本里
    // 把 New Version 作为当前 Version。
    Finalize(v); // 更新Size Compaction的统计信息
    AppendVersion(v); // 安装新版本成为当前版本
    manifest_file_number_ = next_file;
    next_file_number_ = next_file + 1;
    last_sequence_ = last_sequence;
    log_number_ = log_number;
    prev_log_number_ = prev_log_number;

    // 检查是否可以继续使用当前的 MANIFEST 文件。
    if (ReuseManifest(dscname, current)) {
        // No need to save new manifest
    } else {
        // 如果不能继续使用的话, 需要把当前的 MANIFEST 进行保存。
        *save_manifest = true;
    }
} else {
    std::string error = s.ToString();
    Log(options_>info_log, "Error recovering version set with %d records: %s",
        read_records, error.c_str());
}
```

ReuseManifest检查是否可以继续使用当前的 MANIFEST 文件

小疑问: ReuseManifest检查的标准是什么?

VersionSet::Recover函数的实现 (db/version_set.cc)

- ReuseManifest检查的标准

```
bool VersionSet::ReuseManifest(const std::string& dscname,
                               const std::string& dscbase) {
    if (!options_>reuse_logs) {
        return false;
    }
    FileType manifest_type;
    uint64_t manifest_number;
    uint64_t manifest_size;
    if (!ParseFileName(dscbase, &manifest_number, &manifest_type) ||
        manifest_type != kDescriptorFile ||
        !env_>GetFileSize(dscname, &manifest_size).ok() ||
        // Make new compacted MANIFEST if old one is too big
        manifest_size >= TargetFileSize(options_)) {
        return false;
    }

    assert(descriptor_file_ == nullptr);
    assert(descriptor_log_ == nullptr);
    Status r = env_>NewAppendableFile(dscname, &descriptor_file_);
    if (!r.ok()) {
        Log(options_>info_log, "Reuse MANIFEST: %s\n", r.ToString().c_str());
        assert(descriptor_file_ == nullptr);
        return false;
    }

    Log(options_>info_log, "Reusing MANIFEST %s\n", dscname.c_str());
    descriptor_log_ = new log::Writer(descriptor_file_, manifest_size);
    manifest_file_number_ = manifest_number;
    return true;
}
```

```
static size_t TargetFileSize(const Options* options) {
    return options->max_file_size;
}
```

```
include > leveldb > options.h > {} leveldb > Options
12     namespace leveldb {
34     struct LEVELDB_EXPORT Options {
116         size_t max_file_size = 2 * 1024 * 1024;
```

综合来说，就是比大小。

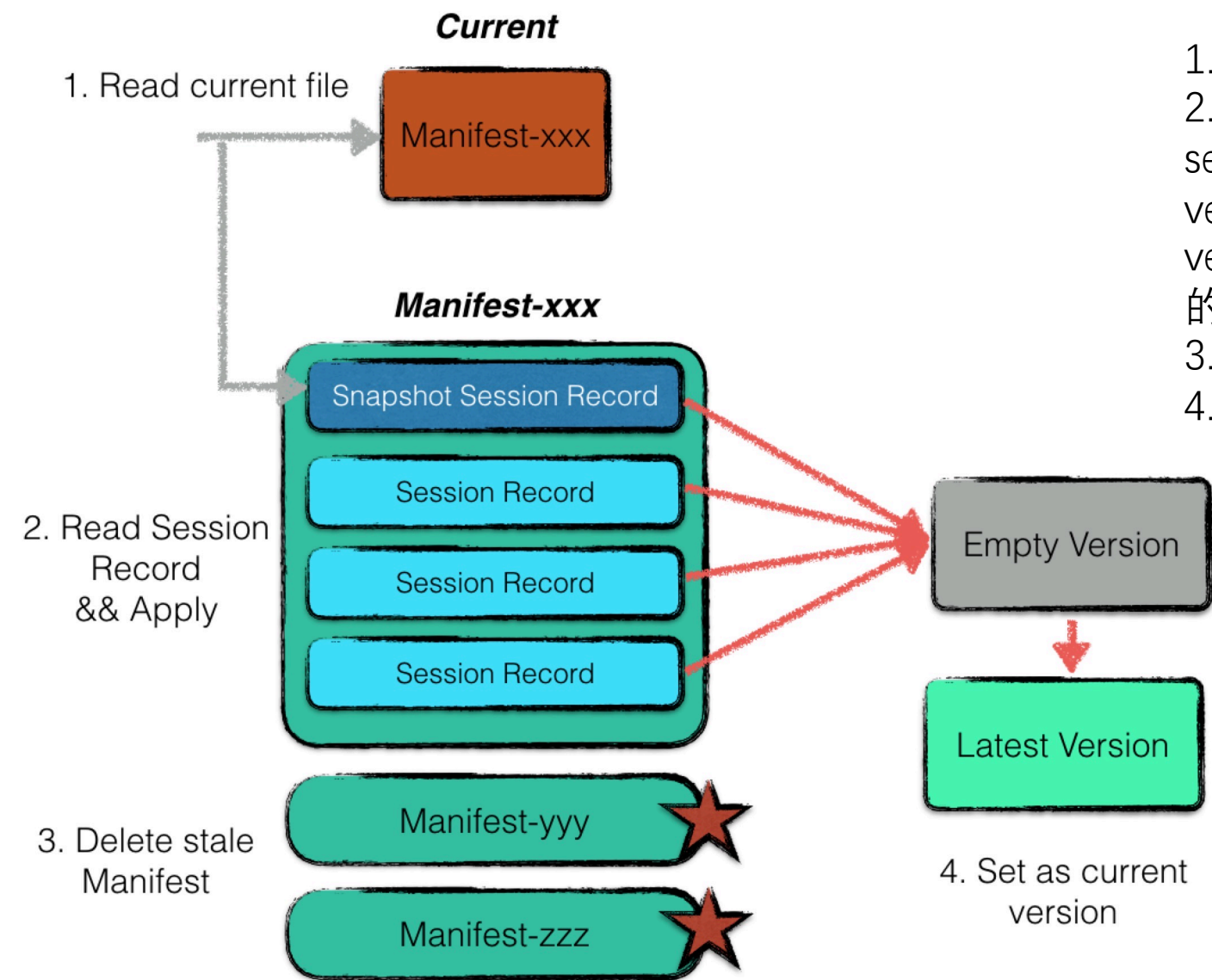
如果MANIFEST文件的大小大于2M，那么就不会重用，而是将当前状态写入到一个新的MANIFEST文件里

新疑问：这种重用方式有什么好处？

避免打开的时候读取太大的MANIFEST，使得打开时间太长。

回看Recover

数据库每次启动时，都会有一个recover的过程，简要地来说，就是利用Manifest信息重新构建一个最新的version



1. 利用Current文件读取最近使用的manifest文件;
2. 创建一个空的version，并利用manifest文件中的session record依次作apply操作，还原出一个最新的version，注意manifest的第一条session record是一个version的快照，后续的session record记录的都是增量的变化;
3. 将非current文件指向的其他**过期**的manifest文件删除;
4. 将新建的version作为当前数据库的version

CURRENT文件的作用

- 整体梳理一下流程

今天的讲解：启动数据库时，通过Current文件加载Manifest文件，读取Manifest文件完成版本信息恢复

更新与Compaction的进行，LevelDB会不断增删文件，需要一个文件来记录文件列表，这个列表就是manifest文件。由于每次启动，都会新建一个Manifest文件，因此leveldb当中可能会存在多个manifest文件。DB需要知道最新的manifest，必须将manifest准备好后原子切换，这就是CURRENT文件的作用。

CURRENT 文件中存储的是当前正在使用的 MANIFEST 文件。当创建新的 MANIFEST 文件时，LevelDB 会先更新 CURRENT文件，使其指向新的 MANIFEST 文件，然后再将旧的 MANIFEST 文件删除。

若没有 CURRENT 文件，新 MANIFEST 文件创建后，还没来得及删除旧的 MANIFEST 文件。假如系统此时崩溃了，那么 LevelDB 恢复时就不知道当前正在使用的 MANIFEST 文件是哪个，也就无法恢复到正确的状态。

进一步的疑问：manifest文件丢失，还能否修复吗？

- 当leveldb的manifest文件丢失时，所有版本信息也就丢失了，但是本身的数据文件还在。
- leveldb提供了Repairer接口供用户进行版本信息恢复。具体恢复的过程：

- 1.按照文件编号的顺序扫描所有的sstable文件，获取每个文件的元数据以及最终数据库的元数据（sequence number等）；
- 2.将所有sstable文件视为0层文件（由于0层文件允许出现key重叠的情况，因此不影响正确性）；
- 3.创建一个新的manifest文件，将扫描得到的数据库元数据进行记录

这个方法确实能保证修复，但是问题在于：效率低下
（需要对整个数据库的文件进行扫描；因为0层文件数量太多导致合并次数过大）

Q&A